

Curvas elípticas en criptografía

Trabajo Fin de Grado - Ingeniería Informática

Leon Elliott Fuller

Julio de 2026

Tutor: Jesús García Miranda

E.T.S. de Ingenierías Informática y de Telecomunicación

Universidad de Granada



UNIVERSIDAD
DE GRANADA

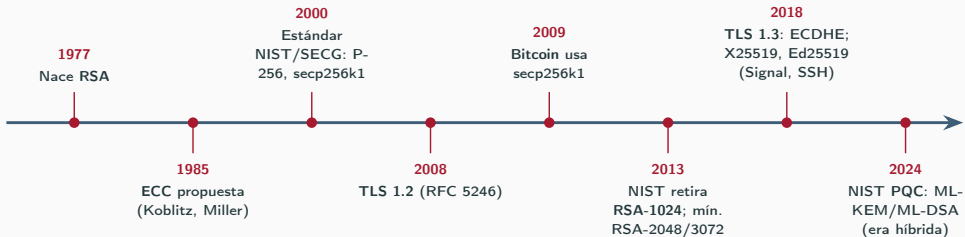


1. Motivación y objetivos
2. Fundamentos: curvas elípticas
3. Protocolos: ECDH, ECDSA y RSA
4. Metodología experimental e implementación
5. Resultados: los tres ejes
6. Conclusiones y trabajo futuro

Motivación y objetivos

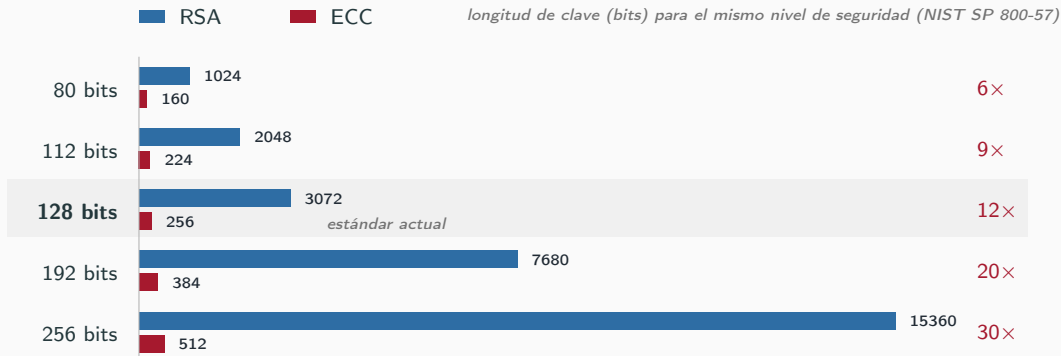
Motivación

- RSA (años 70) sigue siendo el pilar de la clave pública, pero exige claves **cada vez más grandes**.
- **1985**: Koblitz y Miller proponen, de forma independiente, las **curvas elípticas** (ECC): misma seguridad con claves **mucho menores**.



En 40 años ECC ha pasado de propuesta teórica a sostener TLS, Bitcoin y las *passkeys*. ¿Por qué desplazó a RSA? La respuesta está en el **tamaño de las claves**...

El problema, en una imagen: bits necesarios a igual seguridad



RSA: factorización → ataques **subexponenciales** ⇒ la clave se dispara. **ECC:** ECDLP → mejor ataque **exponencial**, $O(\sqrt{n})$ ⇒ crece lineal.

La ventaja es **estructural** y **crece** con el nivel de seguridad: de 6× a 30×.

Objetivo y los tres ejes de comparación

Objetivo: implementar ambos criptosistemas y compararlos empíricamente con rigor sin sesgos en un entorno reproducible.

Eje 1

RSA vs ECC

a igual nivel de seguridad
(NIST SP 800-57)

Eje 2

Afines vs Jacobianas

efecto de las coordenadas
proyectivas

Eje 3

$\mathbb{F}_p$ vs $\mathbb{F}_{2^m}$

cuerpos primos vs binarios

Fundamentos: curvas elípticas

¿Qué es una curva elíptica?

El conjunto de soluciones de una ecuación cúbica sobre un cuerpo finito, más un **punto del infinito** $\mathcal{O}$.

Cuerpos primos $\mathbb{F}_p$

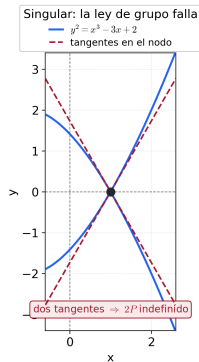
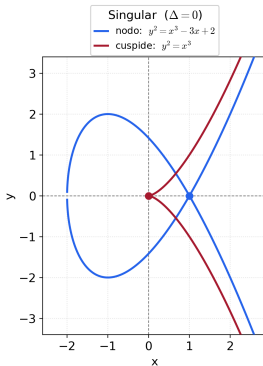
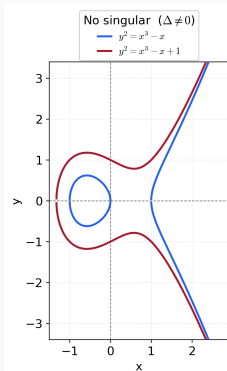
$$y^2 = x^3 + ax + b$$

secp256k1 (Bitcoin), NIST P-256, NIST P-384

Cuerpos binarios $\mathbb{F}_{2^m}$

$$y^2 + xy = x^3 + ax^2 + b$$

5 curvas SEC 2: 3 Koblitz (sect*k1) + 2
aleatorias (sect*r1)

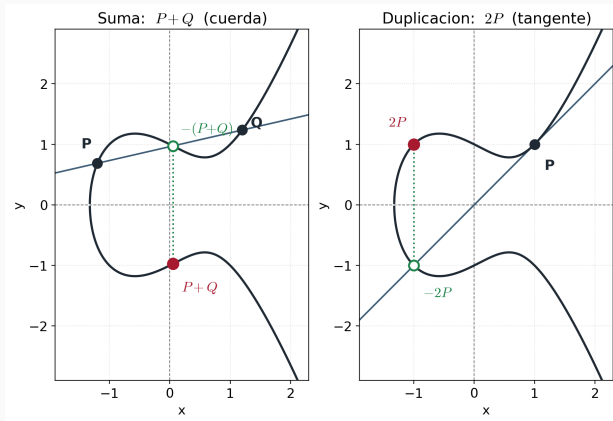


La ley de grupo: cómo se “suman” puntos

Los puntos forman un **grupo abeliano** $(E(K), +)$:

- **Suma** $P + Q$: la recta por P y Q corta la curva en un tercer punto; su **reflexión** es $P + Q$.
- **Duplicación** $2P$: igual, con la **tangente** en P .
- **Neutro**: $\mathcal{O}$ (rectas verticales).

Son las **operaciones atómicas**: la **multiplicación escalar** kP es repetirlas (*double-and-add*).



El grupo $(E(K), +)$: estructura y orden

Grupo abeliano. Para todo P, Q, R :

- Clausura: $P + Q \in E(K)$
- Asociatividad:
 $(P + Q) + R = P + (Q + R)$
- Neutro: $P + \mathcal{O} = P$
- Inverso: $P + (-P) = \mathcal{O}$
- Conmutatividad: $P + Q = Q + P$

Orden $\#E(\mathbb{F}_q)$ (número de puntos):

Teorema de Hasse

$$|\#E(\mathbb{F}_q) - (q + 1)| \leq 2\sqrt{q}$$

En cripto: subgrupo cíclico de **orden primo n** , con $\#E = h \cdot n$ (cofactor h pequeño). La dificultad del ECDLP es $\sim \sqrt{n}$.

Suma y duplicación: coordenadas afines

Cuerpo primo $y^2 = x^3 + ax + b$

Suma ($P \neq Q$):

$$x_3 = \frac{(y_2 - y_1)^2}{(x_2 - x_1)^2} - x_1 - x_2$$

$$y_3 = \frac{(y_2 - y_1)(x_1 - x_3)}{x_2 - x_1} - y_1$$

Duplicación ($2P$):

$$x_3 = \frac{(3x_1^2 + a)^2}{4y_1^2} - 2x_1$$

$$y_3 = \frac{(3x_1^2 + a)(x_1 - x_3)}{2y_1} - y_1$$

Cuerpo binario $y^2 + xy = x^3 + ax^2 + b$

Suma ($P \neq Q$):

$$x_3 = \frac{(y_1 + y_2)^2}{(x_1 + x_2)^2} + \frac{y_1 + y_2}{x_1 + x_2} + x_1 + x_2 + a$$

$$y_3 = \frac{(y_1 + y_2)(x_1 + x_3)}{x_1 + x_2} + x_3 + y_1$$

Duplicación ($2P$):

$$x_3 = \frac{(x_1^2 + y_1)^2}{x_1^2} + \frac{x_1^2 + y_1}{x_1} + a$$

$$y_3 = x_1^2 + \left(\frac{x_1^2 + y_1}{x_1} + 1 \right) x_3$$

Cada operación necesita una **división** (inversión modular) en el cuerpo.

Suma y duplicación: coordenadas Jacobianas (cuerpo primo)

Punto como clase $(X:Y:Z)$ con $x = \frac{X}{Z^2}$, $y = \frac{Y}{Z^3}$: la Z absorbe el denominador y **no se invierte en cada paso**.

Suma ($P \neq Q$):

$$U_1 = X_1 Z_2^2, \quad U_2 = X_2 Z_1^2,$$

$$S_1 = Y_1 Z_2^3, \quad S_2 = Y_2 Z_1^3,$$

$$H = U_2 - U_1, \quad R = S_2 - S_1:$$

$$X_3 = R^2 - H^3 - 2U_1 H^2$$

$$Y_3 = R(U_1 H^2 - X_3) - S_1 H^3$$

$$Z_3 = H Z_1 Z_2$$

Duplicación ($2P$):

$$A = Y_1^2, \quad B = 4X_1 A,$$

$$C = 8A^2, \quad D = 3X_1^2 + aZ_1^4:$$

$$X_3 = D^2 - 2B$$

$$Y_3 = D(B - X_3) - C$$

$$Z_3 = 2Y_1 Z_1$$

Suma: $12M + 4S$ · Duplicación: $4M + 4S$ · Sin inversiones

¿Por qué las Jacobianas son más eficientes?

- La **inversión modular** es la operación más cara del cuerpo.
- Modelo clásico: $I \approx 80\text{--}100 M$ (multiplicaciones).
- Recuento de una *duplicación*:
 - Afín: $1I + 2M + 2S$
 - Jacobiana: $4M + 6S$ (**sin I**)
- Si $I \gg M$, ganan las Jacobianas: **una sola inversión al final**.

Conexión con los benchmarks

La teoría predice $\sim 8\times$; nosotros medimos $\sim 1.8\times$.

NTL/GMP hacen la inversión tan rápida que el cociente real I/M es mucho menor que 80–100, y el ahorro se reduce.

Multiplicación escalar y el problema difícil

Multiplicación escalar: $kP = \underbrace{P + P + \dots + P}_{k \text{ veces}}$, calculada con **double-and-add** en $O(\log k)$ pasos.

Fácil

Dados k y P , calcular $Q = kP$.

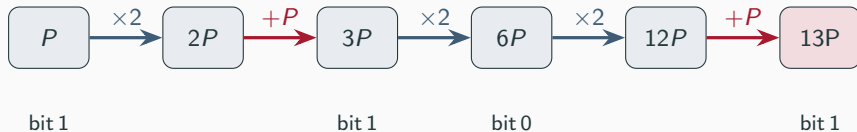
Inviabile (ECDLP)

Dados P y $Q = kP$, hallar k .

Esta asimetría es el **Problema del Logaritmo Discreto en Curvas Elípticas (ECDLP)**.
Sobre él descansan **ECDH** y **ECDSA**: k es la clave privada, $Q = kP$ la pública.

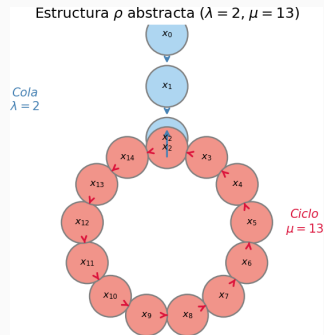
Double-and-add: cómo se calcula kP

Se recorren los **bits de k** de izquierda a derecha; en cada bit se **duplica** ($\times 2$) y, si el bit vale 1, se **suma P** . Ejemplo $k = 13 = (1101)_2$:



$13P$ en 5 operaciones (3 duplicaciones + 2 sumas), no 12 sumas: en general $\sim \log_2 k$ pasos
 $\Rightarrow O(\log k)$.

Atacar el ECDLP: el algoritmo rho de Pollard



Paseo **pseudoaleatorio** con cada punto escrito como $R_i = a_iP + b_iQ$; por la **paradoja del cumpleaños** llega la **colisión** $R_i = R_j$ y se despeja $k \equiv (a_i - a_j)(b_j - b_i)^{-1} \pmod{n}$

Coste: $O(\sqrt{n})$ tiempo, $O(1)$ memoria $\Rightarrow$ **exponencial** en bits.

Protocolos: ECDH, ECDSA y RSA

Parámetros del dominio

Para instanciar ECC sobre $\mathbb{F}_p$, todas las partes acuerdan la séxtupla

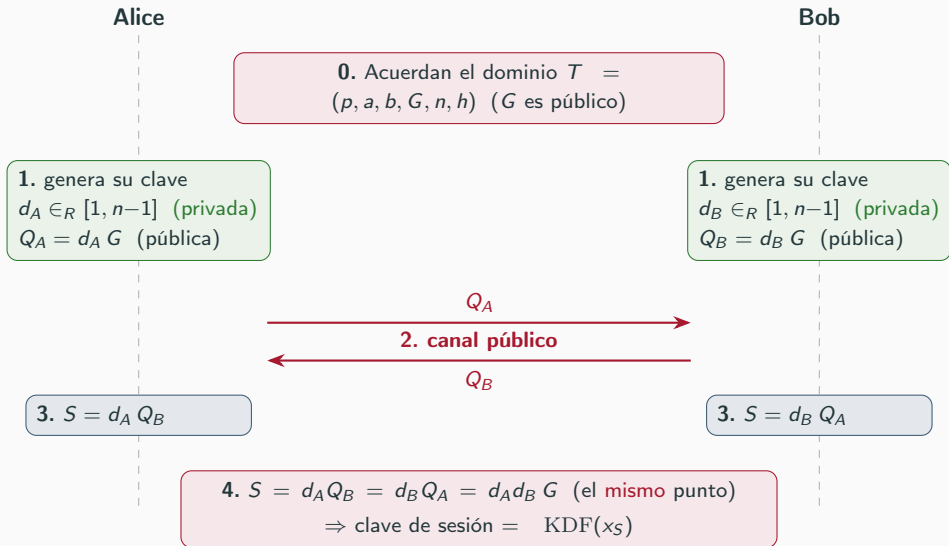
$$T = (p, a, b, G, n, h)$$

Requisitos de seguridad

- p : primo que define $\mathbb{F}_p$.
- a, b : coeficientes de $y^2 = x^3 + ax + b$.
- $G = (G_x, G_y)$: punto generador.
- n : orden de G (menor con $nG = \mathcal{O}$).
- $h = \#E(\mathbb{F}_p)/n$: cofactor.
- n primo (evita Pohlig–Hellman).
- h pequeño, ideal 1 (evita curva inválida).
- No anómala: $\#E(\mathbb{F}_p) \neq p$.
- Grado de inmersión alto (evita MOV).
- $\Delta = -16(4a^3 + 27b^2) \neq 0$.

En el TFG usamos curvas estándar **NIST/SECG**, ya auditadas por la comunidad criptográfica.

Intercambio de claves: ECDH paso a paso



Firma digital: ECDSA paso a paso

Firmante (Alice): privada d

1. $e = H(m)$
2. nonce $k \in_R [1, n-1]$ (secreto, único)
3. $R = kG = (x_R, y_R)$, $r = x_R \bmod n$
4. $s = k^{-1}(e + dr) \bmod n$

$\Rightarrow$ firma $\boxed{(r, s)}$

$m, (r, s)$

Verificador (Bob): pública $Q = dG$

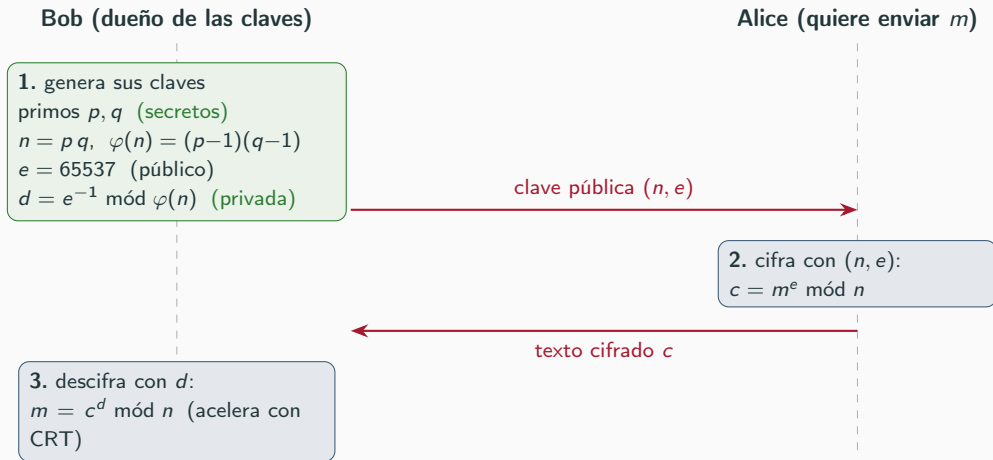
1. $e = H(m)$, $w = s^{-1} \bmod n$
2. $u_1 = ew$, $u_2 = rw \bmod n$
3. $R' = u_1G + u_2Q$
4. aceptar si $x_{R'} \bmod n = r$

Por qué cuadra: $k = u_1 + u_2 d \Rightarrow$
 $R' = (u_1 + u_2 d)G = kG = R$

El nonce k es crítico

Reutilizarlo o predecirlo $\Rightarrow$ se despeja la clave privada d (casos PlayStation 3 y Bitcoin). Solución: nonce determinista (RFC 6979).

El contraste: RSA paso a paso



verde = secreto, azul = público. Romper RSA = factorizar n (subexponencial) $\Rightarrow$ claves grandes.

Metodología experimental e implementación

Elección del *stack*: C++17 + NTL/GMP

Lo elegido

- **C++17**: control total; sin recolector de basura ni intérprete que contaminen los tiempos.
- **NTL 11.5.1 sobre GMP 6.3.0**: la referencia en aritmética de precisión arbitraria (ZZ_p para $\mathbb{F}_p$, GF2E para $\mathbb{F}_{2^m}$).
- OpenSSL 3.0.2: **solo** como baseline externo de contraste.

Lo descartado

- **Python / Sage**: esconden justo la aritmética que se quiere estudiar y medir.
- **Rust**: alternativa moderna razonable, pero con un ecosistema de multiprecisión menos maduro que NTL/GMP.
- **Construir sobre OpenSSL**: mediríamos su ensamblador a mano, no los algoritmos.

1. Medición precisa

- Enemigos: ruido del SO, cachés frías, *outliers*
- 3 calentamientos descartados
- $N = 100$ medidas (`std::chrono`, μs)
- Mediana como métrica principal

2. Entorno determinista

- Docker: Ubuntu 22.04, GMP/NTL desde fuente
- `g++ -O2` fijo, sin `-march=native`
- Ejecución monohilo

3. Comparar con las mismas entradas

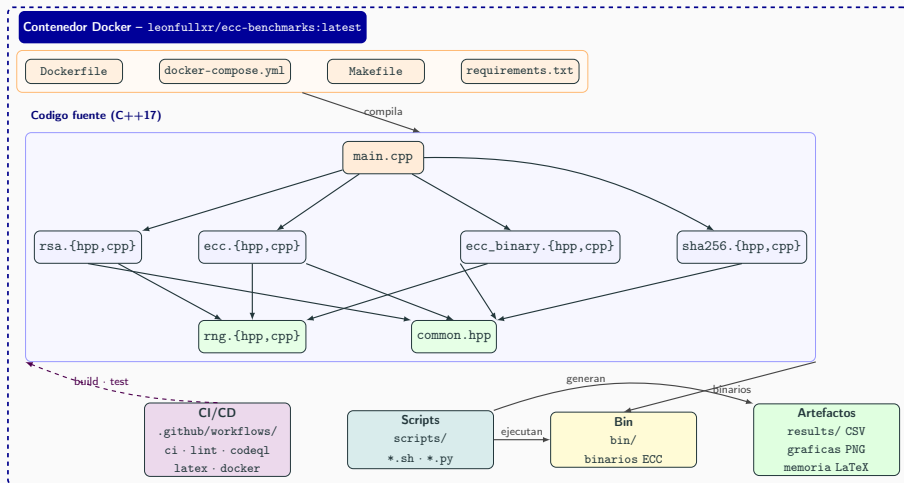
- Semilla fija ($s=0$) $\Rightarrow$ comparación pareada
- RNG validado: χ^2 , K-S, rachas, autocorrelación, entropía

En cifras

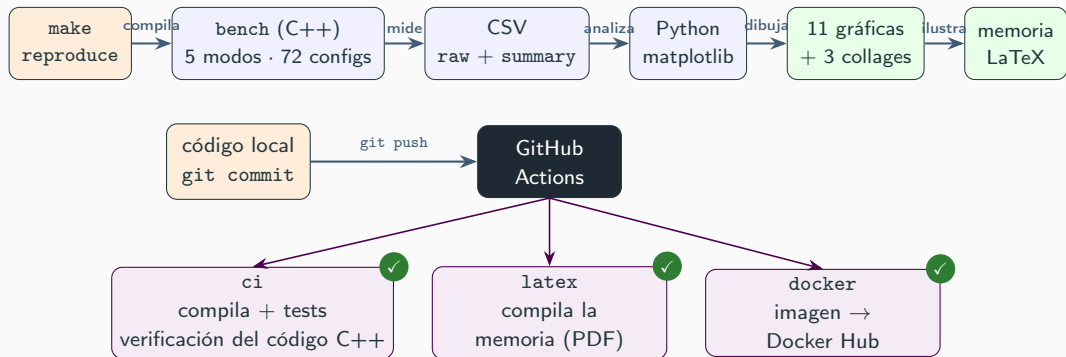
72 configuraciones medidas (24 RSA + 18 ECC afín + 15 Jacobiano + 15 binario)

Hardware: AMD Ryzen 9 7950X, 64 GB RAM

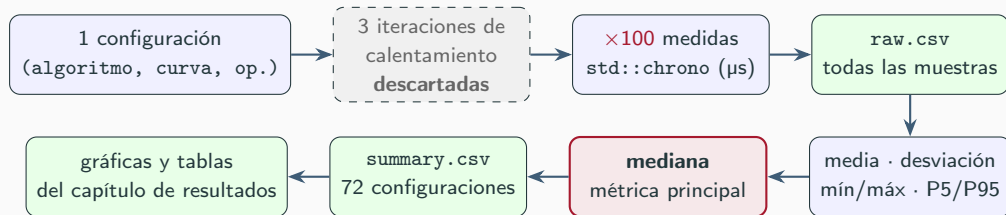
Arquitectura del proyecto



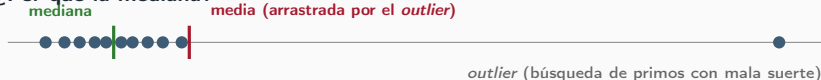
Orquestación: todo el capítulo de resultados, un comando



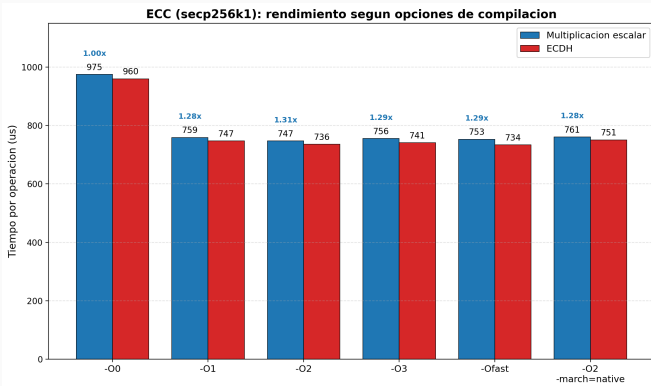
El flujo estadístico: de 100 medidas a una gráfica



¿Por qué la mediana?



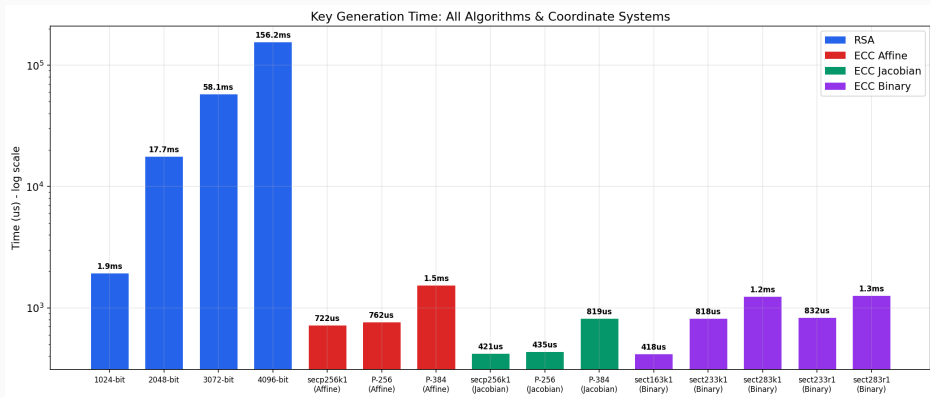
Experimento previo: elección de las opciones de compilación



De `-O0` a `-O1` se gana $\sim 1.25\times$; por encima de `-O2` las mejoras son marginales (el cálculo pesado vive en GMP, ya precompilado) y `-O3` puede introducir ruido. Se fijó `-O2` como estándar de compilación del TFG.

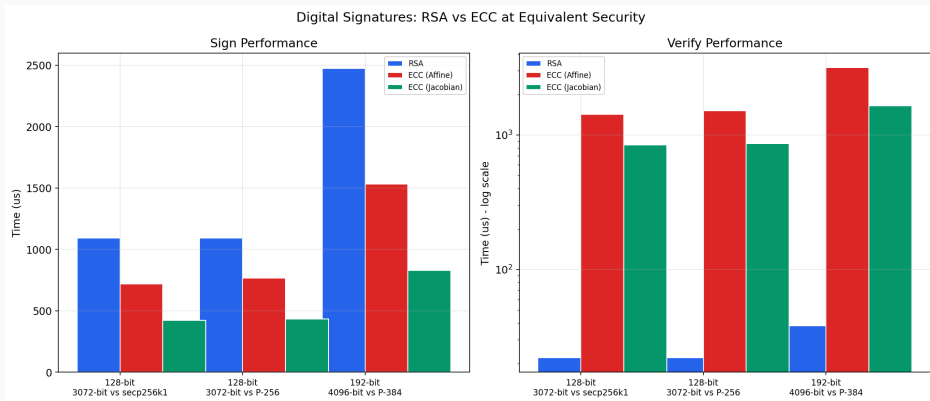
Resultados: los tres ejes

Eje 1 - RSA frente a ECC: generación de claves



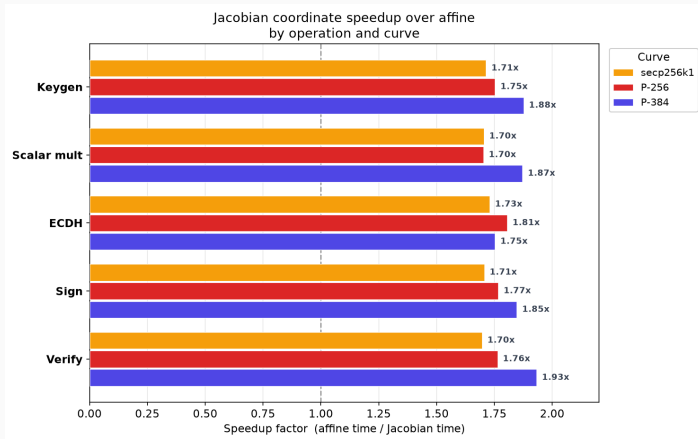
RSA debe **buscar primos grandes** por ensayo y error; ECC solo **elige un escalar**. RSA-3072 ≈ 58 ms frente a ECC Jacobiano ≈ 0.42 ms: $\sim 138\times$ (escala logarítmica).

Eje 1 - RSA frente a ECC: firma y verificación



Firma: ECC $\sim 2.6\times$ más rápida · **Verificación:** RSA $\sim 38\times$ más rápida (exponente público $e = 65537$, solo 2 bits a 1).

Eje 2 - Coordenadas afines vs Jacobianas

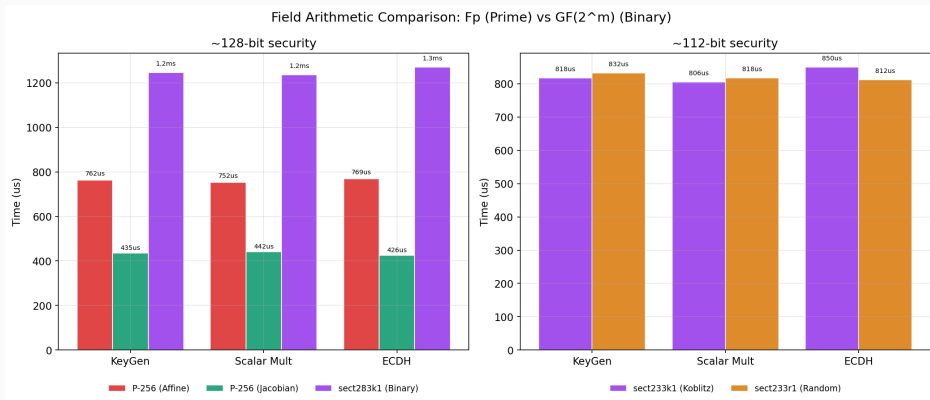


- Las Jacobianas **evitan la inversión modular**.
- Medido: **1.70–1.93×** (media $\sim 1.8\times$).

La brecha ES el hallazgo

La teoría predice $\sim 8\times$;
NTL/GMP hacen la inversión barata, y el ahorro real es mucho menor.

Eje 3 - Cuerpos primos vs binarios



A 128 bits el binario es $\sim 1.7\times$ más lento que el primo (ambos en afín): GF2E genérico frente al ensamblador de GMP. Es un artefacto de software, no del álgebra.

Validación: contraste con OpenSSL

	OpenSSL vs propia
P-256 (operaciones)	14–34× más rápido
P-384	1.7–4× más rápido
RSA-4096 verificación	más lento (0.92×)

Por qué la diferencia

Ensamblador a mano + **blinding** contra canales laterales que mi versión académica no incorpora.

En RSA verify, la excepción: sin padding OAEP/PSS ni las capas EVP de OpenSSL, la versión propia es algo más rápida.

Objetivo del TFG: **entender y comparar**, no competir con código de producción.

Conclusiones y trabajo futuro

Conclusiones: ECC frente a RSA

ECC — cuándo elegirla

Pros: claves pequeñas ($256\text{ b} \approx \text{RSA-3072}$); generación de claves y *firma* rápidas; menos ancho de banda, memoria y energía.

Ideal para: TLS 1.3 efímero, móvil/IoT, *passkeys*, Signal/WhatsApp, Bitcoin/Ethereum (secp256k1).

RSA — cuándo sigue ganando

Pros: *verificación* muy rápida ($e = 65537$); esquema simple y ubicuo; enorme base instalada y compatibilidad.

Ideal para: PKI/certificados heredados, *tokens*/JWT verificados muchas veces, sistemas legados de larga vida.

No hay ganador absoluto: la ventaja de ECC en tamaño de clave es **estructural** (ECDLP exponencial) y **crece** con la seguridad; RSA conserva la ventaja en **verificación**.

El valor del trabajo: separar el álgebra de la ingeniería.

Trabajo futuro

- Padding seguro: **OAEP** (cifrado) y **PSS** (firma) en RSA.
- Optimizaciones: **Lopez-Dahab** en cuerpos binarios.
- **Horizonte post-cuántico**: Shor rompe **RSA y ECC a la vez**.
NIST (2024): ML-KEM, ML-DSA, SLH-DSA (retículos y hashes).

Recorrido

El banco de pruebas construido es **directamente reutilizable** para comparar los esquemas post-cuánticos frente a ECC.

Gracias por su atención

`github.com/leonfullxr/ECC-Thesis`

Respaldo - de dónde sale el $\sim 8\times$ teórico (y el $1,8\times$ real)

Conteo para kP de 256 bits (≈ 256 duplicaciones + 128 sumas), con $S \approx 0,8M$:

- **Afín** ($1I + 2M + 2S$ por paso): $\approx 384 I + 1280 M$ -equivalentes.
- **Jacobiana** (dupl. $4M + 4S$, suma $12M + 4S$, +1 inversión final): $\approx 3900 M$ -equivalentes.

Con el coste de libro ($I \approx 80M$)

Afín $\approx 32\,000 M \Rightarrow$ razón $\approx 8\times$

Con el coste real de GMP ($I \approx 14M$)

Afín $\approx 6\,600 M \Rightarrow$ razón $\approx 1,7\times$

Las Jacobianas no eliminan coste: **intercambian** 1 inversión por ~ 10 multiplicaciones extra por paso. El intercambio solo es muy rentable si I es cara, y NTL/GMP la hacen barata (I/M real ~ 10 – 20). Por eso el ahorro **crece** con el tamaño del cuerpo ($1,72\times$ en 256 bits $\rightarrow \sim 2\times$ en P-384).

Respaldo - de la ecuación general a la forma corta

Partimos de la **ecuación general de Weierstrass** ($\text{char}(K) \neq 2, 3$):

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6.$$

Paso 1 — completar el cuadrado en y (usa $\text{char} \neq 2$). El miembro izquierdo es $y^2 + (a_1x + a_3)y$; con $y \mapsto y - \frac{1}{2}(a_1x + a_3)$ se anula el término lineal en y y queda

$$y^2 = x^3 + \frac{b_2}{4}x^2 + \frac{b_4}{2}x + \frac{b_6}{4}, \quad b_2 = a_1^2 + 4a_2, \quad b_4 = 2a_4 + a_1a_3, \quad b_6 = a_3^2 + 4a_6.$$

Paso 2 — eliminar el término en x^2 (usa $\text{char} \neq 3$). Para el cúbico $x^3 + px^2 + qx + r$ (aquí $p = \frac{b_2}{4}$, $q = \frac{b_4}{2}$, $r = \frac{b_6}{4}$), la traslación $x \mapsto x - \frac{p}{3} = x - \frac{b_2}{12}$ lo *deprime*:

$$\boxed{y^2 = x^3 + ax + b}, \quad a = \frac{b_4}{2} - \frac{b_2^2}{48}, \quad b = \frac{b_6}{4} - \frac{b_2b_4}{24} + \frac{b_2^3}{864}.$$

Las divisiones por 2 (paso 1) y por 3 (paso 2) son exactamente la razón de exigir $\text{char}(K) \neq 2, 3$. El discriminante resulta $\Delta = -16(4a^3 + 27b^2)$.

Respaldo - corrección de la verificación ECDSA

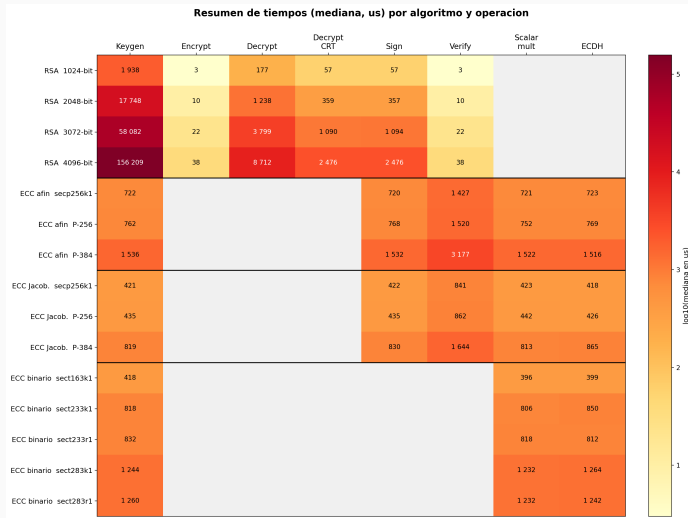
Si la firma es honesta, $s = k^{-1}(e + d r)$ mód n , luego

$$k = s^{-1}(e + d r) = \underbrace{s^{-1}e}_{u_1} + \underbrace{s^{-1}r}_{u_2} d = u_1 + u_2 d \quad (\text{mód } n).$$

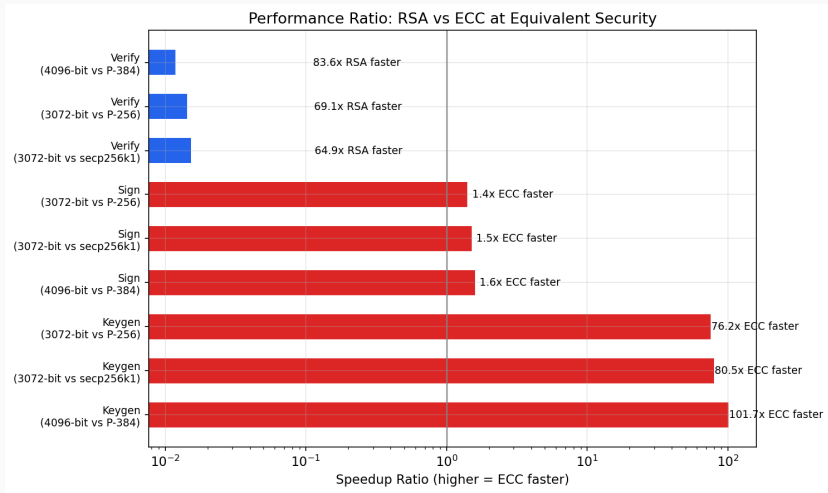
Por tanto $R' = u_1 G + u_2 Q = u_1 G + u_2 d G = (u_1 + u_2 d) G = k G = R$, de donde $x_{R'} \text{ mód } n = x_R \text{ mód } n = r$ y la firma se acepta.



Respaldo - panorama completo de tiempos



Respaldo - razón de rendimiento RSA/ECC



Respaldo - CRT en el descifrado RSA

Se calcula $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, $q_{inv} = q^{-1} \bmod p$. Recombinación de Garner:
 $m_1 = c^{d_p} \bmod p$, $m_2 = c^{d_q} \bmod q$, $h = q_{inv}(m_1 - m_2) \bmod p$, $m = m_2 + hq$. Acelera $\approx 3.5\times$ (dos exponenciaciones a la mitad de bits).

Respaldo - ECC hoy en día: ¿dónde se usa?

- **Web (TLS 1.3):** ~97–98 % de los handshakes usan ECDHE; **X25519** por defecto en navegadores.
- **Passkeys / FIDO2:** firmadas con **ECDSA P-256**.
- **Mensajería y redes:** Signal, WhatsApp, SSH, WireGuard → **Curve25519 / Ed25519**.
- **Criptomonedas:** Bitcoin y Ethereum → **secp256k1** (la curva de Koblitz del TFG).
- **Documentos:** pasaportes electrónicos (ICAO Doc 9303).

La transición post-cuántica es *híbrida*, no una retirada

X25519MLKEM768 (X25519 + ML-KEM-768) ya cubre >40 % del tráfico HTTPS humano en Cloudflare, por defecto en Chrome, Firefox, OpenSSL y Apple. X25519 sigue como **salvaguarda clásica**.

Respaldo - la asociatividad de la ley de grupo

$(E(K), +)$ es grupo abeliano. Neutro $\mathcal{O}$, inversos (reflexión) y conmutatividad son inmediatos. El axioma no trivial es la **asociatividad**: se prueba con el teorema de Cayley–Bacharach o, más elegantemente, identificando E con su grupo de clases de divisores $\text{Pic}^0(E)$ (Riemann–Roch).